Silent Failures in LLM-Generated CUDA Kernels

Ali Ansari

Dissertation report submitted for the MSc Dissertation Module,

School of Computing Sciences, University of East Anglia, 2025/26

Abstract

This project separates CUDA kernels that are genuinely correct from those that only pass the benchmark. KernelBench scores a generated kernel against a reference PyTorch module on five random, contiguous inputs. That test is kept, at a tighter tolerance, and stress tests are added that change the memory layout of the inputs, their size and type, and repeat the call. Kernels that pass the benchmark but fail under stress are counted as silent failures. The stress tests are hidden from the repair loop, so the model cannot learn to satisfy them. Applied to 100 problems across three models, with reasoning on and off on the same weights, and on two GPUs, this yields 2,852 kernel attempts. Repair raises the share the benchmark accepts in every condition, by up to 43 points, and the share of those that are silently wrong with it. Of the 244 kernels repair turns into a benchmark success, 79 are silent failures, twice as often as among kernels accepted first time, 25.7% against 12.4%. After repair, KernelBench detects no significant difference between the three models, while the stress tests still separate them. A repaired kernel is not reliably more correct; it is reliably harder to catch.

Acknowledgement

I would like to thank my supervisor, Prof. Stephen Laycock, for encouraging me throughout this project and for the guidance that helped me see it through.

I am also grateful to Dr Jeannette Chin for her help and guidance along the way.

Finally, I thank my family for their constant support and encouragement, which carried me through a demanding year.

Statement of AI Usage

Claude was used only as a research-support tool to help me understand background information and explore literature on similar or related topics.

Contents

1 Introduction **7**

1.1 Motivation 7

1.2 Aim and objectives 8

1.3 Terminology 8

1.4 Contributions 9

1.5 Report structure 10

2 Literature Review **10**

2.1 Writing code and checking it 10

2.2 How KernelBench decides that a kernel is correct 11

2.3 Tests do not measure correctness; they define it 12

2.4 Repair loops and the signal they learn from 13

2.5 The gap 14

3 Design **15**

3.1 Research design 15

3.2 A taxonomy of kernel failures 16

3.3 The classifier 16

3.4 The stress battery 19

3.5 The repair loop 21

3.6 Experimental plan 22

3.7 Metrics and units of analysis 23

4 Implementation **24**

4.1 System architecture 24

4.2 Execution environment 25

4.3 Caching and provenance 26

4.4 Validation 26

4.5 Reproducibility 27

5 Evaluation **27**

5.1 Dataset and scale 27

5.2 Reasoning and the failure distribution 28

5.3 Repair relocates failures 29

5.4 Repair and model ranking 33

5.5 Which probe does the work 34

5.6 Cross-GPU divergence 35

5.7 Performance of passing kernels 35

6 Limitations **36**

7 Conclusion **37**

7.1 Summary of findings 37

7.2 Against the objectives 38

7.3 Future work 38

References **39**

A Generation and Repair Prompts **42**

B One Worked Example per Category **44**

1 Introduction

1.1 Motivation

Machine learning depends on a small amount of code that almost nobody writes. A GPU kernel is the layer where a mathematical operation becomes instructions on hardware, and writing one well takes expert knowledge of memory hierarchies and thread scheduling. FlashAttention, one of the most consequential kernels of the last decade, arrived five years after the architecture it accelerates (Dao et al., 2022; Ouyang et al., 2025).

This makes kernel generation an obvious target for language models, and the field has moved quickly (Li et al., 2026b; Dai et al., 2026). The systems considered here share a component and a blind spot. The component is a repair loop: when a generated kernel fails, the error is returned to the model and another attempt is requested, and this reliably raises benchmark scores. The blind spot is what counts as failure. A kernel is judged by comparing its output against a reference implementation on a few randomly drawn inputs, so a kernel that is wrong in a way those inputs do not reach is recorded as correct.

One example shows what that allows. A PyTorch module is handed to a model, which returns a CUDA kernel meant to replace it. The benchmark draws a few random tensors, runs both versions, finds the outputs match, and records the kernel as correct. Now give that kernel the same numbers arranged differently in memory, as a slice of a larger tensor rather than a compact block. PyTorch handles this without complaint. The kernel, which computed each memory address by assuming the numbers were laid out end to end, reads the wrong elements and returns the wrong answer. Nothing crashes and no error is printed. The benchmark never tried that input, so its verdict does not change. This dissertation calls such a kernel a *silent failure*.

Put those two facts together and an uncomfortable possibility appears. If a model is repeatedly asked to fix the failures a test can see, it may learn to produce kernels whose failures the test cannot see, and the benchmark would report progress either way.

This dissertation asks whether that happens. The question matters because a kernel is a drop-in replacement for a component that already worked. A loud failure costs an engineer an afternoon; a silent one returns plausible numbers for as long as nobody checks.

1.2 Aim and objectives

The aim of this project is to determine whether iterative repair of LLM-generated CUDA kernels improves their correctness, or merely conceals their faults from the benchmark that scores them.

The objectives are:

1. Build a classifier that separates kernels which satisfy KernelBench's correctness check from those which are correct under stronger testing, and validate it against kernels of known category.
2. Design a stress battery of metamorphic probes that can be withheld from the model, so that a repair loop cannot be optimised against it.
3. Measure the false-pass rate before and after repair, across several models, both reasoning settings and two GPU architectures.
4. Determine whether repair changes how the benchmark ranks models.
5. Release the classified dataset and the analysis code so that every reported number can be reproduced.

This aim is narrower and sharper than the one in the original proposal, which set out to build a general taxonomy of correctness failures. The taxonomy was built, and it appears in Section 3.2, but it turned out to be a means rather than an end: the interesting result was not the distribution of failure types but the movement between them under repair. The aim is stated here as the project actually developed.

1.3 Terminology

Several terms in this report are either borrowed from software testing or introduced here, and are used from Chapter 2 onwards. They are collected below so that no definition arrives after its first use. The five failure categories have their own table in Section 3.2 and are not repeated here.

Reference module The PyTorch implementation a generated kernel is asked to replace. It defines what the kernel should compute, because no other specification exists.

Oracle The procedure that decides whether a test has revealed a fault; KernelBench's oracle compares outputs on five random inputs. Every oracle is incomplete, because it can only inspect the inputs it is given.

Probe, stress battery A probe transforms the input, runs both the reference module and the candidate on the result, and reports whether they still agree. The battery is the full set of probes, run only after a kernel has satisfied KernelBench's own check.

Arm One of the two experimental conditions: *think* with reasoning enabled, *direct* with the same model and prompt and reasoning disabled.

Turn, chain A turn is one request to the model, turn 0 being the first attempt and turns 1 to 3 repairs. A chain is the sequence of turns for one problem, in one arm, on one GPU.

Visible failure A failure KernelBench can see, namely `syntactic`, `runtime` or `functional`; these are the failures reported back during repair.

Silent failure A kernel that satisfies KernelBench's check and is nonetheless wrong, because a probe disagrees with the reference module.

Relocation A repair turn that converts a visible failure into a silent one. The benchmark records it as a successful repair, because the failure it could see has gone.

1.4 Contributions

Chapter 2 reviews how kernel generation is evaluated and how repair loops are used, and closes in Section 2.5 with the gap this work addresses: repair loops are optimised against a correctness check known to be incomplete, and no published work measures what repair does to the kernels that survive it. Addressing that gap, this project makes the following contributions.

1. A withheld stress battery for generated kernels. KernelBench's correctness check is extended with seven kinds of probe over memory layout, magnitude, dtype and repeated execution, and their results are kept out of the repair prompt so that the model cannot fit them.

2. Evidence that repair relocates failures. Across three models and two GPU architectures, iterative repair raises the acceptance rate while also increasing the share of accepted kernels that

are silent failures. Of the 244 chains that repair turns into an apparent success, 79, almost one in three, are silent failures.

3. Evidence that repair reduces the benchmark's discriminating power. Under KernelBench's own contract the three models converge after repair to between 91.5% and 93.5%, and no statistically significant pairwise difference is detected, while the extended reading produces a wider spread and a different ordering.

4. A quantified account of where the effect comes from. 85% of silent failures are caught by one relation, that a kernel must not change its answer when the same values arrive through different strides. Because that relation extends beyond KernelBench's stated contract, strict and extended correctness are distinguished, and the two are compared directly where the distinction changes a conclusion.

5. An accompanying dataset and a reproducible analysis pipeline. 2,852 classified kernel attempts, 1,196 repair chains and the generated sources, from which every figure and table in this report can be regenerated.

1.5 Report structure

Chapter 2 reviews how kernel generation is evaluated, why test suites define correctness rather than measure it, and what is known about repair loops, and ends with the gap this work fills. Chapter 3 sets out the experimental design, the failure taxonomy, the stress battery and the repair loop. Chapter 4 describes the implementation and how it was validated. Chapter 5 reports the results, Chapter 6 sets out the limitations, and Chapter 7 concludes.

2 Literature Review

2.1 Writing code and checking it

Large language models write useful code. Codex demonstrated this first, and it also fixed how the field measures success: generate a program from a description, run the tests that come with the task, and report the fraction that pass (Chen et al., 2021). That method works because the task carries its own definition of correctness. The docstring states what the function should do, and the tests check whether it does it.

CUDA kernels break this arrangement in two ways.

The task is set in PyTorch, a widely used machine learning framework (Paszke et al., 2019). A PyTorch module describes a computation as a sequence of high-level operations, such as a convolution followed by an activation, and the framework calls a library routine for each. Replacing that module with a CUDA kernel means writing the same computation directly against the GPU.

The first difficulty is widely recognised. A kernel is written against a particular memory hierarchy and thread model, so good performance depends on hardware knowledge that the source code never states (Kirk and Hwu, 2012). FlashAttention, an attention kernel that reduces traffic between levels of GPU memory, required years of expert effort (Dao et al., 2022), and needed two further years to reach a new generation of GPUs (Ouyang et al., 2025).

The second difficulty attracts less attention, and it is the subject of this dissertation. A kernel has no docstring. It is not written against a specification that a test can consult. It is written to replace an existing PyTorch module, so the only available definition of correct is "behaves like that module". Comparing two implementations on every possible input is impossible. In practice somebody selects a small number of inputs and compares the outputs on those.

The choice of inputs therefore decides what correctness means.

2.2 How KernelBench decides that a kernel is correct

KernelBench is the standard benchmark for this task (Ouyang et al., 2025). It supplies 250 PyTorch modules across three difficulty levels and asks the model to return an equivalent module built around a custom CUDA kernel. It then reports fast$_p$: the share of generated kernels that are correct and at least p times faster than PyTorch. Frontier reasoning models beat the PyTorch baseline in fewer than 20% of cases, so the benchmark is far from solved.

The correctness half of that score rests on a single procedure. The authors "evaluate on 5 random inputs per problem", checking "equivalence of output between reference architecture `Model` and generated architecture with custom kernel `ModelNew`". In that notation `Model` is the original PyTorch module and `ModelNew` is the generated replacement built around a CUDA kernel. They are called the reference module and the candidate throughout.

They explain the choice openly. Proving two programs equivalent is undecidable, so they adopt random testing, which they describe as "the most common practical approach". They argue that it suits this domain, because AI kernels have simple control flow and "the focus is primarily on

numerical correctness". Five inputs are presented as "a good tradeoff between the ability to catch errors and efficiency". To justify that number they ran an experiment on 100 generated kernels, and found that kernels either passed everything or failed everything: "the 0/5 and 0/100 failures indicate that no partial correctness was observed".

That experiment is where the argument weakens. Every input in it shares one shape and one memory layout, and only the values inside the tensors change. A kernel whose fault depends on layout rather than on values will therefore pass all five draws, and all one hundred. The absence of partial correctness follows from the way the inputs were sampled. It is not evidence that five inputs are sufficient.

Two recent papers approach this point without reaching it. Lange et al. (2025) report that existing kernel benchmarks "suffer from exploitable loopholes and insufficient diversity in testing conditions", and respond with a stricter benchmark of their own. Li et al. (2026a) show that a model can satisfy the tests "with repair by degeneration, simplifying the CUDA code into a safer but slower program". Both establish that the score can be gamed. Neither asks the simpler question: how often is the score merely wrong?

2.3 Tests do not measure correctness; they define it

Software engineering has a name for the difficulty reached at the end of the last section. A test oracle "determines whether a test execution reveals a fault, often by comparing the observed program output to the expected output", and building a complete one is "recognised as one of the fundamental challenges of software testing" (Segura et al., 2016). KernelBench's five inputs are an oracle in exactly this sense. The useful question is not whether the oracle is imperfect, since its authors say as much, but how much it misses.

Liu et al. (2023) measured that for Python. They took HumanEval and increased its test cases by a factor of eighty, then re-evaluated 26 models on the extended version. The additional tests caught "significant amounts of previously undetected wrong code" and reduced pass@k by up to 19.3–28.9%.

The more troubling result concerns the ranking. Two models that lost to ChatGPT on the original benchmark defeated it on the extended one. A weak oracle does not flatter every model by the same amount; it reorders them.

Metamorphic testing offers a disciplined way to add tests when the correct answer is not known in advance. Instead of checking a single output, it transforms the input and observes how the output "morphs" in response (Segura et al., 2016). For a kernel offered as a drop-in replacement, one such relation is almost too obvious to state: the same values, presented through a different memory layout, must produce the same result. None of the kernel benchmarks reviewed here applies it.

Ludwig et al. (2026) show what happens when somebody applies it in a neighbouring setting. Testing LLM-translated CUDA-to-OpenMP code, they found faults in programs "that compile and pass developer-written tests yet produce incorrect scientific results", and report that these faults "are often input-dependent". Their subject is translation rather than generation, and they use no repair loop, but the failure mode is the one measured here.

That failure mode already has a name outside software testing. Studies of large server fleets describe corruptions in which "the only symptom is an erroneous computation" (Hochschild et al., 2021). The same word, silent, is adopted here for a kernel that passes the benchmark and returns the wrong answer.

2.4 Repair loops and the signal they learn from

If a program fails, show the model the error and ask again. Chen et al. (2024) formalised this idea, teaching a model to inspect execution results and explain its own code, and reported improvements of up to 12% on benchmarks that provide unit tests. Madaan et al. (2023) generalised it to any task, using a single model as generator, critic and reviser, and reported roughly 20% absolute improvement across seven tasks. Results of this kind are why repair is now a standard component rather than an experiment.

Three later studies complicate the picture.

Olausson et al. (2024) charged repair its real cost. Each turn is another model call, so the fair comparison is against simply drawing more candidates. Judged that way, the gains are "often modest, vary a lot between subsets of the data, and are sometimes not present at all". They locate the reason in the feedback itself: repair is limited by "the model's ability to provide feedback on its own code", and improves sharply when a stronger model supplies that feedback.

Huang et al. (2024) go further. Without external feedback, models "struggle to self-correct their responses", and at times become worse. They then explain where earlier positive results

came from. Those experiments used oracle labels to guide the correction, and "the improvements vanish when oracle labels are not available".

Brown et al. (2024) arrive at a compatible conclusion from a different direction. Repeated sampling steadily raises the chance that some attempt is correct, across four orders of magnitude. That coverage converts into real performance only "in domains like coding and formal proofs, where answers can be automatically verified".

Taken together, these three make one point. A repair loop can be no better than the judge it answers to.

The kernel generation systems adopt repair without revisiting that judge. CUDA-L1 rewards speedup through reinforcement learning, Astra runs several agents through generation, testing and profiling, Kernel-Smith evolves a population of candidates using "structured execution feedback on compilation, correctness, and speedup", and CUDA Agent trains against "automated verification and profiling to provide reliable reward signals" (Li et al., 2026b; Wei et al., 2025; Cao et al., 2026; Du et al., 2026; Dai et al., 2026; Chen et al., 2025). Reasoning models, which work through intermediate steps before answering (Wei et al., 2022), perform best throughout.

Across these systems the feedback is the same short list: whether the code compiled, whether it crashed, how fast it ran, and whether it matched the reference on the standard inputs. None of them feeds back anything about memory layout.

2.5 The gap

Two lines of work meet in this dissertation, and they have not met before.

The recent CUDA kernel systems reviewed here all repair, returning compiler errors and failed comparisons to the model and asking again. Software testing has argued for decades that such comparisons are incomplete oracles, and that adding tests to a benchmark exposes code that was wrong from the start.

Lange et al. (2025) showed that KernelBench's oracle can be exploited. Li et al. (2026a) showed that a model can satisfy it by making the kernel worse. Neither measured what repair does to correctness across several turns.

So the question remains unasked. When a model repairs its own kernel, does the kernel become correct, or only harder to catch? That is what this project measures.

3 Design

3.1 Research design

Section 2.4 ended with a prediction. A repair loop can be no better than the judge it answers to, and every kernel system answers to a judge that Section 2.3 shows to be incomplete. This chapter describes an experiment built to test that prediction.

Testing that prediction requires three things, and no existing harness provides them together.

First, a correctness check stronger than the benchmark's. Without one, a wrong kernel cannot be told from a right one.

Second, that stronger check must be hidden from the model. If the model can see it, the model will repair against it.

Third, the same problem must be followed through a repair chain, with every kernel version recorded, so that any change can be attributed to repair.

The design below provides all three. KernelBench's check is kept in form as the benchmark's verdict, with one tightened tolerance that Section 3.3 states and Section 6 returns to, a battery of further checks is added and never shown to the model, and a repair loop runs driven only by that verdict, recording every intermediate kernel.

KernelBench is used rather than a stricter benchmark such as `robust-kbench` (Lange et al., 2025) for the same reason. The weak oracle is the object of study, not an obstacle to it, and replacing it with a stronger one would remove the very thing whose consequences this work measures.

One further control shapes the whole study. Comparing a reasoning model with a non-reasoning model would confuse reasoning with everything else that differs between two checkpoints: training data, size, tokeniser, alignment. This is avoided by running two arms on the same model, using the same prompt and the same temperature. The only difference is the `reasoning_effort` parameter, set to `high` in one arm and `none` in the other. The extremes are used rather than intermediate settings so that the contrast is as large as the parameter allows. This isolates the reasoning setting while avoiding checkpoint-level confounds.

This contrast is stronger than the paired-checkpoint comparison used in most earlier work. It is possible only because current models expose reasoning as a runtime switch over fixed weights (Qwen Team, 2025).

3.2 A taxonomy of kernel failures

KernelBench reports one bit, correct or not, and that bit hides distinctions which matter for diagnosis. Every generated kernel is therefore classified into one of five categories, shown in Table 1.

The categories are ordered, and the first one that matches wins. This makes each label deterministic, because a kernel that fails to build is never also tested for numerical accuracy. The order runs from the most visible failure to the least visible, which is also the order in which a developer would meet them.

category	meaning
`syntactic`	does not build
`runtime`	crashes, trips the sanitizer, or is too slow to evaluate
`functional`	runs, wrong answer on the standard input
`silent`	passes the standard check, wrong under the stress battery
`pass`	correct on the standard input and survives the battery

Table 1: The five failure categories. The first matching stage wins, so every kernel receives exactly one label.

`silent` is the category this dissertation exists to measure. A silent kernel compiles, runs, and agrees with the reference module on the inputs KernelBench draws. Under KernelBench it is a success and counts towards fast_p, and it is nonetheless wrong.

The other four categories exist mainly so that `silent` can be separated cleanly from them. The rest of this chapter explains how each label is assigned.

Two further labels appear in the results and fall outside this taxonomy, because they describe the attempt rather than the kernel: `empty` marks a response that contained no code block, and `error` a fault in the harness. Neither is counted as an accepted kernel, and together they account for under 2% of attempts.

3.3 The classifier

The classifier runs a fixed sequence of stages. Algorithm 1 gives the sequence, and the rest of this section explains three decisions inside it.

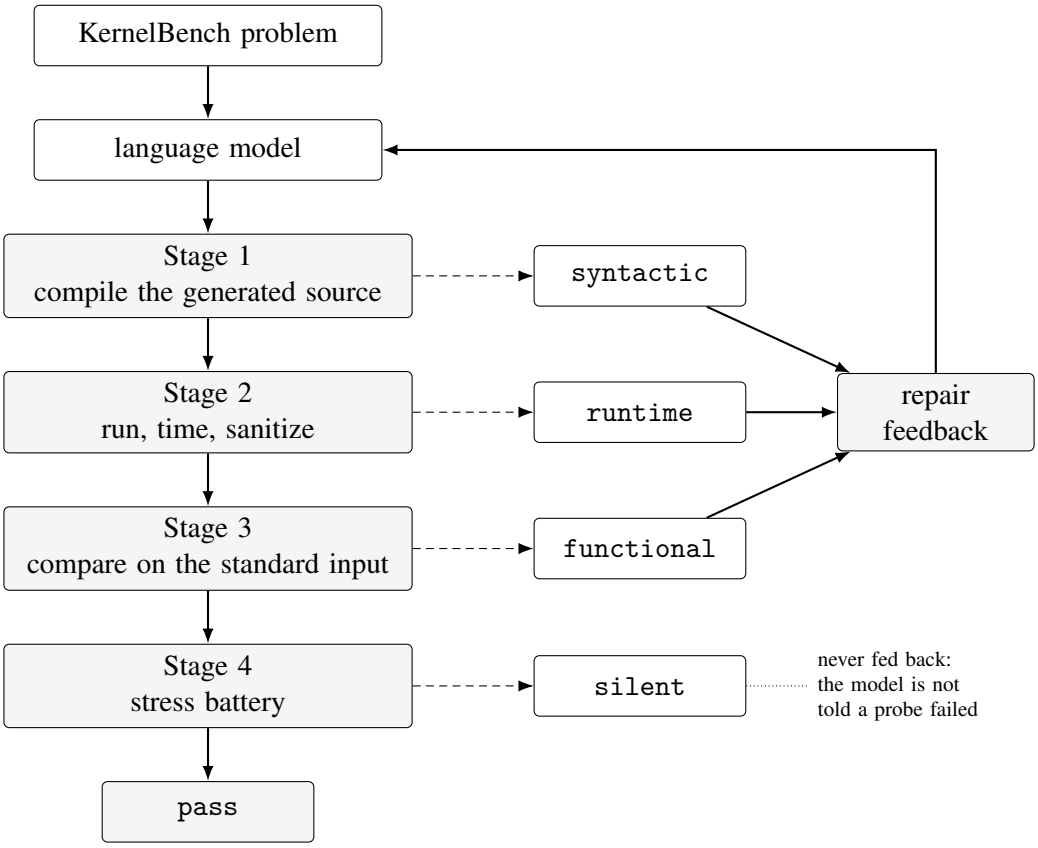


Figure 1: The classification pipeline and the repair loop. A generated kernel passes through four stages, and the first stage it fails determines its category. Visible failures are returned to the model, which tries again for up to three further turns. Stage 4 is deliberately excluded from that feedback, so the model can never repair against the stress battery.

Algorithm 1 Classifying one generated kernel

```
seed the generator; build the reference module from the problem
compile the generated source                                  ▷ Stage 1: does it build?
if compilation fails then return syntactic
seed again; construct the candidate module with identical weights
run one forward pass on the standard input, then synchronise  ▷ Stage 2a: does it run?
if it raises then return runtime
time one warm forward call                                    ▷ Stage 2b: can it be evaluated?
if it exceeds the forward-time limit then return runtime
run compute-sanitizer                                         ▷ Stage 2c: does it violate memory?
if a violation is reported then return runtime
compare candidate and reference output                        ▷ Stage 3: right on the standard input?
if they disagree then return functional
run the stress battery                                        ▷ Stage 4: right under stress?
if any probe disagrees then return silent
return pass
```

Weight synchronisation. Many Level 2 problems contain learnable parameters. If the reference module and the candidate module are built from different random draws, they hold different weights. Their outputs then differ for reasons that have nothing to do with the kernel, and every kernel looks wrong. Following KernelBench, the random generator is reseeded three times: before the initialisation inputs are produced, before each module is constructed, and before every input draw. The two modules then hold identical weights and see identical data.

A limit on forward time. A full classification issues roughly ninety forward calls, so a kernel taking tens of seconds per call cannot be evaluated at all. Stage 2b times one warm call and labels such a kernel `runtime`, rather than letting it exhaust the container and disappear from the dataset unlabelled. The call is timed warm because the first also pays for module loading. The limit is raised on slower hardware, so that no kernel is rejected merely for running on a slower device.

The sanitizer stage did not run. Stage 2c invokes NVIDIA's `compute-sanitizer` to detect memory violations that do not crash the program. The stage is implemented and is exercised by the validation suite, but it reports that the device is unsupported on both GPUs available for this work, so it contributed no labels to the results in Chapter 5. Races and non-crashing out-of-bounds reads are therefore undetected throughout, and Section 6 returns to what that costs.

A comparison tolerance. Outputs are compared with an absolute and relative tolerance of 10^{-3}, applied recursively through tuples, lists and dictionaries so that structured returns are handled. The tolerance is a real choice: set it too tight and ordinary floating-point reassociation is reported as a fault; set it too loose and a genuinely wrong kernel passes.

The value used here is tighter than the 10^{-2} that KernelBench states for its own tasks. A kernel whose output differs from the reference by between the two thresholds is therefore labelled `functional` here, while a standard KernelBench harness would accept it. The acceptance rates in Chapter 5 are consequently not identical to those KernelBench itself would report, and Section 6 returns to what that means.

3.4 The stress battery

The battery is the stronger oracle promised in the research design. Because it carries the study's central measurement, it is worth restating plainly what it is before the individual probes are described.

The battery is a fixed set of additional tests that runs only after a kernel has already satisfied KernelBench's own check at stage 3. A kernel that fails earlier never reaches it, so the battery is applied exactly to the population the benchmark would have declared correct. Each test, called a probe, derives a new input from the standard one, runs both the reference module and the candidate on that input, and records whether the two still agree. A kernel that survives every probe is labelled `pass`; a kernel that fails any probe is labelled `silent`. The outcome is recorded but never shown to the model, for the reason given in Section 3.5.

Two properties of that design matter. Each probe presents inputs the standard check never uses, and it compares the candidate against the reference on those same inputs rather than against a stored expected answer. A probe can therefore only fail when the two implementations disagree. The battery never holds its own idea of the right answer, so it cannot punish a kernel for anything except differing from the module it claims to replace. This also means the battery inherits the reference module as its definition of correct: if PyTorch and the kernel agree on a transformed input, the probe passes, whatever that answer happens to be.

Each probe pairs a transformation of the input with a check on the output. The transformations follow the metamorphic approach described by Segura et al. (2016), in which new test inputs are derived from an existing one rather than invented from scratch. The check is differential: the

reference module and the candidate are both run on the transformed input, and the candidate must agree with the reference. Ludwig et al. (2026) combine the same two techniques for translated HPC code.

The transformations differ in what they preserve, and the distinction matters. Restriding a tensor, or repeating an identical call, leaves the correct output unchanged, so those two probes are metamorphic relations in the strict sense: the candidate must return exactly what it returned before. The remaining probes change the input values, so the correct output changes with them, and the invariant is only that the candidate continues to match the reference. Table 2 lists the seven kinds of probe. They amount to nine individual checks, because the fresh-draw probe is run three times. The seven were chosen to cover the properties a kernel might silently depend on but which the standard input merely happens to supply: the particular values drawn, their layout in memory, their magnitude, their element type, and the absence of state carried between calls. The set is not exhaustive, and Section 6 treats that as a limitation.

probe	transformation	fault it exposes
`fresh_seed_1..3`	redraw the input values	hardcoded or lucky outputs
`non_contig`	restride without changing values	assuming dense row-major layout
`nondeterministic`	repeat the identical call	races, reused internal buffers
`zeros`	all-zero input	division by zero, uninitialised reads
`small_magnitude`	scale values down	numerical instability
`extreme`	scale values up	overflow, saturation
`fp16`	halve the input precision	dtype assumptions

Table 2: The stress battery. Every probe holds the reference implementation fixed, and requires the candidate to agree with it on the transformed input.

One probe deserves separate comment, because it dominates the results. `non_contig` builds a tensor holding exactly the same values as the standard input, but presented through non-unit strides. It does this by stacking the tensor with itself and taking one slice, a construction chosen because it alters the strides while leaving the shape and every value untouched; a transpose or a permutation would also reorder the data, which would confound layout with order. A kernel that indexes raw pointers, assuming a dense row-major layout, then reads the wrong elements and returns a wrong answer. A kernel that calls `.contiguous()`, or respects the strides it is given, returns the right one. KernelBench draws only contiguous tensors, so this fault is invisible to it by construction.

Two rules keep the headline number honest.

The first is that a harness fault is never charged to the kernel. If a probe raises because the machine ran out of memory, or because the harness itself failed, the probe records `skipped` and the kernel is not condemned. Otherwise a 16 GB card would convert working kernels into silent failures whenever memory ran short, and inflate the study's central measurement.

The second is that refusing a dtype is not failing it. A kernel is compiled against a specific element type, so one written for 32-bit floats rejects half-precision input before a single thread runs. That is the kernel enforcing the contract it declared, not computing a wrong answer, so it is recorded as a robustness flag rather than a silent failure.

3.5 The repair loop

After a failure the model is shown its own source and the actual error, and asked again. The loop runs for up to three further turns. Three is a compromise between giving repair a fair chance and spending model calls on chains that will not recover, and the results justify it after the fact: of the chains repair rescues, 59% reach a passing kernel on the first repair turn and only 14% on the third, so a fourth turn would add little.

Only `syntactic`, `runtime`, `functional` and `empty` are repairable, and `pass` and `silent` end the chain. The feedback is deliberately plain: the compiler's own message, the exception, or the reported numerical mismatch. Logs are truncated to their last 800 characters, because `nvcc` puts the decisive line last and the output before it is largely repeated build commands.

The most important decision in this design is what the loop is *not* told.

Stage 4 never appears in the feedback. A silent kernel ends its chain in exactly the way a passing kernel does, and the model is never told that a stress probe disagreed.

This is not a convenience. Huang et al. (2024) show that reported gains from self-correction came from oracle labels guiding the process, and that they disappear once those labels are withdrawn. A battery fed back into the prompt would be exactly such a label: the model would repair against it, the false-pass rate would fall, and the fall would measure the model's skill at fitting a held-out test rather than any improvement in correctness. Withholding stage 4 keeps the battery an independent instrument. It also matches deployment, since a developer running a repair loop against KernelBench sees compiler errors and failed comparisons, not a stress battery.

3.6 Experimental plan

Models. Three models are evaluated: GLM-5, DeepSeek V3.2 and Qwen3-235B-A22B (hereafter Qwen3-235B), each served through Amazon Bedrock.

Models were selected for experimental controllability rather than for benchmark performance. The two-arm design needs a model whose reasoning can genuinely be turned off, and this was tested rather than assumed. Several otherwise suitable candidates continued to reason regardless of the parameter, and those were rejected: their `direct` arm would silently have been a second `think` arm, and the central contrast would have been empty. Among the models that honoured the switch, the three chosen come from three different developers, so a result common to all three is less likely to be a property of one training pipeline.

Two further constraints shaped the choice, the first of which is cost. The design issues one model call per problem, per arm, per turn, which came to 1,448 generations and roughly 8.5 million tokens. Frontier proprietary models are priced well above the open-weight models used here, and at this volume that gap separates an experiment that can be run from one that cannot. The question under study is what a repair loop does to a benchmark verdict, not which model writes the best kernels, so paying for the strongest available models would have raised the cost without changing what is measured.

The second constraint is uniformity. Serving all three models through a single provider means one client library, one prompt format and one set of parameters, so no difference between models can be attributed to the harness. Proprietary frontier models such as Claude and GPT were considered and set aside on both grounds: each exposes reasoning through its own interface, which would have introduced a second source of variation alongside the one under study, and each is substantially more expensive per token at this scale. This restricts generality, and it is stated here as a limitation. The findings apply to models that expose reasoning as a runtime switch.

Tasks. KernelBench Level 2, the fusion tier, in which several PyTorch operators must be combined into one kernel. Level 1 is excluded because single-operator problems are largely solved and produce too few failures to analyse. Level 3 is excluded for time, and Section 6 returns to that exclusion. All 100 Level 2 problems are used, so no sampling decision can bias which failures are observed.

Hardware. Every problem runs on two GPUs: an Ada-generation L4 and a Turing-generation T4. Both use one pinned CUDA image and one pinned set of Python packages. Running both makes it possible to detect a kernel that passes on one architecture and fails on another, and guards against a result that is an artefact of a single device.

Determinism. Temperature is zero, and one sample is drawn per problem, per arm, per turn. Each cell of the design is therefore a single greedy generation. The design consequently measures no variation between repeated generations of the same problem, and Section 6 states what this excludes. The benchmark dataset is pinned to one revision, so later upstream edits cannot change what was run.

The full design is 3 models $\times$ 2 arms $\times$ 2 GPUs $\times$ 100 problems $\times$ 4 turns. It is evaluated lazily: a chain stops as soon as it reaches `pass` or `silent`.

3.7 Metrics and units of analysis

Three different quantities are easy to confuse, so each is named separately and used consistently from here on.

Acceptance rate The share of kernels KernelBench would report as correct, which is the `pass` and `silent` categories together. This is the number a standard harness prints.

Full-battery pass rate The share of kernels that satisfy KernelBench's check *and* survive every probe, which is the `pass` category alone.

False-pass rate The share of accepted kernels that are silent, written $silent/(silent+pass)$. Equivalently, it is the difference between the acceptance rate and the full-battery pass rate, divided by the acceptance rate. It is the study's central measurement.

Strict and extended correctness. Repeated random draws and the determinism check test only what KernelBench's contract already promises, so a kernel failing them is wrong under any reading. The remaining probes, which restride the input or change the magnitude or type of its values, go beyond that contract. *Strict* correctness condemns a kernel only on the first group; *extended* correctness, which treats the kernel as a drop-in replacement, condemns it on either. Chapter 5 reports false-pass rates under the extended reading, and compares the two readings directly where the distinction changes a conclusion, in Section 5.4.

Unit of analysis. A *chain* is one problem attempted by one model in one arm on one device, and contains up to four *attempts*, one per turn. Rates are computed over chains, using the first attempt for a one-shot figure and the last for a post-repair figure. Where the two GPUs are pooled each problem contributes twice, so pooled and per-device results are reported separately.

Statistical tests. McNemar's test compares two models over the same problems, because those observations are paired. Fisher's exact test compares two disjoint sets of kernels, those accepted at the first attempt against those repair made acceptable. A bootstrap over problems gives an interval for a single rate. None of these measures variation between repeated generations, which the design does not capture.

4 Implementation

4.1 System architecture

The system is organised around one rule: code that decides an outcome must be testable without a GPU. Everything in the pure package therefore makes no API calls, touches no GPU and opens no network connection, and only the top-level scripts talk to the model provider or to Modal, the cloud platform used to obtain GPUs on demand. Table 3 lists the parts.

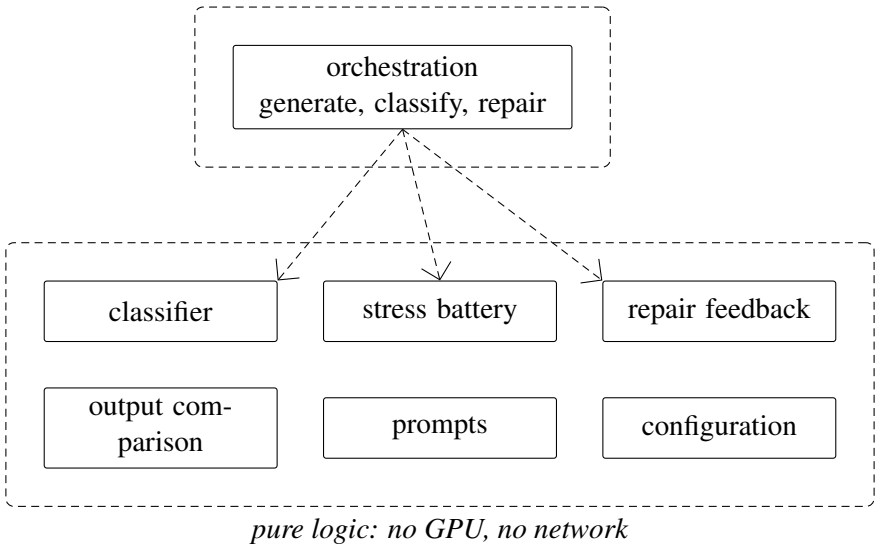


Figure 2: Component and dependency structure. Dependencies run in one direction only: orchestration uses the pure logic, and nothing in the pure logic reaches a GPU, a network or the model provider. This is what allows the classifier, the battery and the repair feedback to be tested without hardware.

component	responsibility
classifier	the stages of Algorithm 1
stress battery	the probes of Table 2
output comparison	comparison across nested return structures
repair feedback	what is reported back, and what is withheld
prompts	prompt construction and response parsing
configuration	every tunable, and the guards between them
orchestration	generation, classification and repair on the GPU platform

Table 3: Responsibilities either side of the boundary. Everything above the rule is pure logic and runs anywhere.

The split exists for one practical reason. Because the classifier and the battery carry no infrastructure, their behaviour can be checked on an ordinary laptop without a GPU. A design that mixed the two would have turned every change to the logic into an experiment.

The orchestration component is the least cohesive part of the system, since it both defines the execution environment and drives the repair chain. Separating the two would be the first structural change to make.

4.2 Execution environment

Generated code is untrusted. A model asked for a CUDA kernel can return anything, and that code is compiled and executed. Each classification therefore runs on Modal in a container, an isolated environment with its own filesystem, that is used once and then destroyed. The network is blocked and access to other resources in the account is withdrawn, so a generated kernel can reach nothing outside its own container and cannot affect the next one.

Reproducibility requires the environment to be fixed rather than merely described. Every run used CUDA 12.4 with the same pinned Python packages, and the environment is identified by a cryptographic digest rather than by a version label, so that a later rebuild cannot silently substitute different software. The Python packages are pinned to exact versions: `torch==2.6.0`, `ninja==1.11.1.1`, `openai==2.41.1`, `datasets==5.0.0` and `modal==1.5.0`. The KernelBench dataset is pinned to one revision, so a later upstream edit cannot silently change the problems.

One setting affects the numbers directly. TF32 arithmetic is disabled for both matrix multiplication and cuDNN, because it reduces precision differently between GPU generations and would

otherwise confound the comparison in Section 5.6. With it disabled, a numerical disagreement is attributable to the kernel rather than to the hardware.

4.3 Caching and provenance

Generations and classifications are cached under independent keys, so a change to the classifier can be applied to kernels already generated. The generation key identifies the request to the model and excludes the GPU, which is why one kernel can be classified on both devices without being generated twice.

One principle governs both keys: *a field that decides an outcome must be part of the key*. The forward-time limit is the clearest case, since it decides whether a slow kernel is labelled `runtime` or proceeds to the battery. Each record also carries the version of the classifier that produced it, so results from a superseded version can be excluded, as Section 4.4 describes.

4.4 Validation

A classifier that assigns the wrong label produces confident and worthless results, so validation runs at two levels. The first is a suite of unit tests over the pure logic, needing no GPU, covering selection and deduplication, chain outcomes, the rate calculations, the dtype and out-of-memory rules in the battery, the repair policy and the cache keys. The second is known-answer validation: eleven hand-written kernels of known category are run through the real classifier on real hardware, covering correct kernels with weights, mutating inputs, structured returns and nested inputs, plus one kernel each for the four failure categories and one that is correct but too slow. The silent case is right on contiguous input and wrong on strided input, so it exercises the probe the results depend on most. The check asserts the category and, where it matters, the failing stage, since `runtime` is reachable several ways.

This layer earned its cost: four defects were found that would each have corrupted the headline number, and all four had passed the unit tests.

The most serious was silent in the strict sense: the routine that copies tensors preserved their values but not their strides, so the `non_contig` probe was handing the kernel a contiguous tensor and testing nothing. Every false-pass rate measured before the fix was measured with the study's main probe disabled. The second was its opposite, an `fp16` probe that reported a crash on every

passing kernel, from which the rule in Section 3.4 follows: a probe firing on 100% of cases measures the harness. The third was environmental, a setting that failed to reach the container, leaving 70 records labelled with the wrong device; the fix was to read the device name on the GPU itself. The fourth was a missing reseed, so reference and candidate held different weights on any problem with learnable parameters and every kernel looked wrong.

4.5 Reproducibility

No figure or table in this report contains a hand-copied number. Every result derives from two exported tables, one row per classified kernel and one row per repair chain, together with the generated kernel sources. The code was developed under Git version control, and the code, the exported tables and the kernel sources are submitted alongside this report as a single archive.

Two kinds of reproducibility should be distinguished. The analysis is exactly reproducible: re-running the scripts over the submitted tables reproduces every figure and table here, and does so without access to a GPU, but the generations are not. Temperature is zero, so the model was asked for a deterministic response, but the provider serves one model through an alias that is repointed to newer checkpoints over time, and the pipeline did not record which checkpoint answered. Repeating the generation step at a later date may therefore query a different model.

5 Evaluation

5.1 Dataset and scale

The experiment produced **2,852 classified kernel attempts** arranged into **1,196 repair chains**. Every model was run on every Level 2 problem, in both arms, on both GPUs, which is 1,200 chains in total. The 2,852 attempts derive from 1,448 distinct generations, because the generation key excludes the GPU (Section 4.1): most kernels are generated once and classified on both devices, and 1,404 were classified on both. Four chains are absent: DeepSeek V3.2 on the T4 for problem 98 in the reasoning arm, and problems 37, 43 and 68 in the direct arm. Each of these was lost to a container failure that returned no classification, so no label of any kind exists for them, and they are excluded from every denominator rather than counted as failures. Table 4 gives the distribution of labels over the attempts that were recorded.

category	attempts	share
`functional`	1,114	39.1%
`syntactic`	611	21.4%
`pass`	582	20.4%
`runtime`	341	12.0%
`silent`	172	6.0%
`empty`	23	0.8%
`error`	9	0.3%
total	2,852	

Table 4: Label distribution over all attempts, including intermediate repair turns. `empty` marks a response containing no code block, and `error` a harness fault rather than a property of the kernel. Neither is counted as an accepted kernel.

Wrong answers outnumber build failures, so writing CUDA that compiles is easier than writing CUDA that computes. The 172 silent attempts are 6% of the total but a much larger share of those KernelBench would accept, which is the subject of Section 5.3.

5.2 Reasoning and the failure distribution

Table 5 reports the acceptance rate and the full-battery pass rate for each model and arm, before and after repair.

			acceptance rate		full-battery pass rate	
model	arm	n	one-shot	repaired	one-shot	repaired
GLM-5	think	200	68.5%	93.5%	58.5%	74.5%
GLM-5	direct	200	49.0%	64.5%	38.5%	47.5%
DeepSeek V3.2	think	199	49.7%	92.5%	41.2%	74.4%
DeepSeek V3.2	direct	197	3.0%	13.7%	2.0%	5.6%
Qwen3-235B	think	200	71.5%	91.5%	66.5%	82.5%
Qwen3-235B	direct	200	13.5%	21.5%	2.0%	7.0%

Table 5: Acceptance rate and full-battery pass rate per model and arm. GLM-5 and Qwen3-235B use $n = 200$ chains per arm; DeepSeek V3.2 uses $n = 199$ in the reasoning arm and $n = 197$ in the direct arm, because four chains were lost to container failures. The acceptance rate is what KernelBench reports; the full-battery rate counts only kernels that also survive every probe. Their difference is the share of all chains ending silent; Table 6 instead expresses those silent failures as a proportion of accepted kernels.

Reasoning substantially increases performance, consistent with what the benchmarks already report (Ouyang et al., 2025; Zhu et al., 2026). Measured by the full-battery pass rate, DeepSeek V3.2 and Qwen3-235B each solve 2% of problems on the first attempt with reasoning disabled, against 41% and 67% with it enabled. This matters here for a methodological reason. For two of the three models the direct arm produces too few accepted kernels for a rate over them to be meaningful, six for DeepSeek V3.2 and twenty-seven for Qwen3-235B at the first attempt, so the primary analysis focuses on the reasoning arm. GLM-5's direct arm is better populated and is reported alongside the others in Table 6, but is read with the same caution.

Figure 3 shows how the whole distribution moves under repair rather than a single aggregate rate.

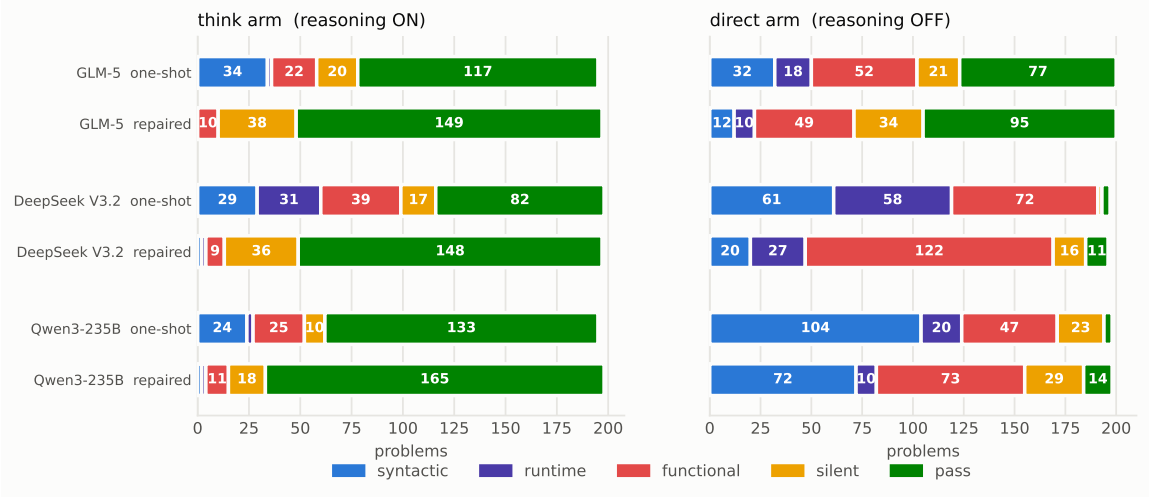


Figure 3: Failure distribution before and after repair, for every model and arm. Repair removes build failures very effectively. The `silent` band does not shrink with them.

5.3 Repair relocates failures

Repair works by the benchmark's own measure: it raises the acceptance rate in every model and every arm, by as much as 43 points for DeepSeek V3.2 in the reasoning arm. By that metric alone, repair would appear uniformly beneficial.

It is not useless by the stronger measure either. The full-battery pass rate, which counts only kernels that also survive every probe, rises in every one of the six conditions in Table 5, and 165 chains are repaired into a genuine pass. Repair does produce correct kernels that a first attempt did not.

The claim of this section is narrower, and worse for the benchmark. The kernels repair adds to the accepted population are silently wrong far more often than those accepted at the first attempt, and the benchmark cannot tell the two apart.

The false-pass rate tells a different story. Table 6 reports, among the kernels KernelBench would accept, the share that fail the stress battery.

model	arm	one-shot	after repair
GLM-5	think	14.6% (20/137)	**20.3%** (38/187)
DeepSeek V3.2	think	17.2% (17/99)	**19.6%** (36/184)
Qwen3-235B	think	7.0% (10/143)	**9.8%** (18/183)
GLM-5	direct	21.4% (21/98)	26.4% (34/129)
DeepSeek V3.2	direct	33.3% (2/6)	59.3% (16/27)
Qwen3-235B	direct	85.2% (23/27)	67.4% (29/43)

Table 6: False-pass rate, with counts, as silent over silent plus pass. In the reasoning arm the rate rises for every model. Direct-arm estimates have small denominators and should therefore be interpreted cautiously.

In the reasoning arm the false-pass rate rises for all three models, by between 2.4 and 5.7 points. Figure 4 shows the movement directly.

The direct arm is less consistent and carries correspondingly less weight. GLM-5 and DeepSeek V3.2 rise as expected, but Qwen3-235B falls, from 85.2% to 67.4%. The fall is arithmetic rather than a reversal of the effect. With reasoning disabled, Qwen3-235B produced only 27 accepted kernels at the first attempt, of which 23 were silent. Repair then added ten genuine passes and six further silent failures, so the count of silent kernels rose while the proportion fell. Rates computed over so few kernels move sharply on a handful of problems, which is why the direct arm is not used to support the argument.

The mechanism is visible in the chains themselves. Counting outcomes across all 1,196 chains, repair rescued 165 chains to a genuine pass and converted **79 chains from a visible failure into a silent one**, which are the relocations. Combining the two gives the number a benchmark would report:

> Repair produced 244 chains that KernelBench scores as a success, of which 79, **almost one in three**, are silent failures.

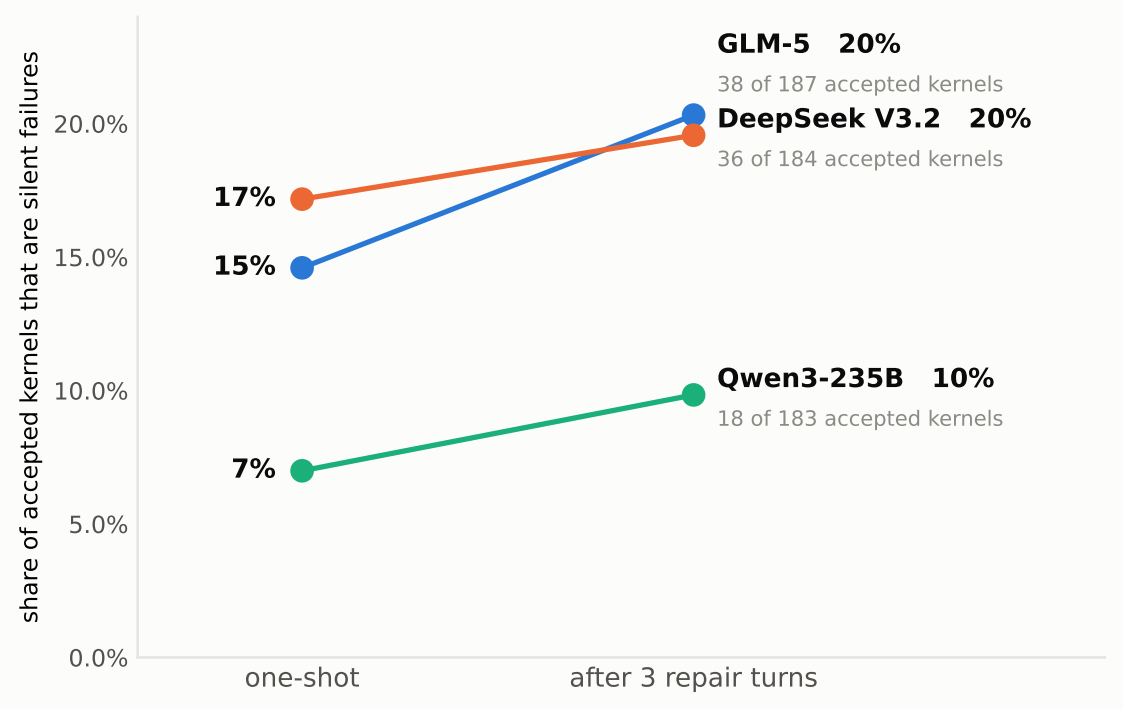


Figure 4: Repair raises the acceptance rate and the false-pass rate together. Every model moves upward, so a harness that reports only the acceptance rate records this as unambiguous improvement.

The effect is not confined to one model. It appears in GLM-5 (32 chains), DeepSeek V3.2 (33) and Qwen3-235B (14), and splits evenly across the two GPUs.

This comparison can be tested directly. Repair produces a set of accepted kernels that is disjoint from the set accepted on the first attempt, so the two can be compared without the shifting denominators that make a before-and-after rate awkward. Fisher's exact test is used because the outcome is binary and some cells are small enough that a chi-squared approximation would be unreliable. The unit of observation is one chain, meaning one problem attempted by one model in one arm on one device. The pooled analysis treats chains as independent, and because problem identities recur across models and devices the pooled values should be read cautiously; per-model and per-device results are reported alongside them. If repair produced work of the same quality as a first attempt, the share of silent failures would be the same in both groups, and it is not. In the reasoning arm, 12.4% of first-attempt acceptances are silent against 25.7% of repair-produced ones, and Fisher's exact test gives $p = 0.0002$. Pooling the two GPUs counts each problem twice, so the test was also run on each device alone, where it holds independently at $p = 0.0058$ on the L4 and $p = 0.0089$ on the T4. Per model the result is significant for GLM-5 ($p = 0.0021$) and

Qwen3-235B ($p = 0.0301$) but not for DeepSeek V3.2 ($p = 0.4567$), whose first attempts were already unusually often silent.

Bootstrapping over the 100 problems gives the false-pass rate itself an interval, and those intervals are wide: 14.6% [6.8, 23.4] rising to 20.3% [12.5, 28.7] for GLM-5, with similar overlap for the other two models. The rise within any single model is therefore not established on its own. The claim this work makes is the one the Fisher tests support, that *the kernels repair produces are more often silently wrong than the kernels accepted on the first attempt*. Figure 5 breaks the chains down by outcome.

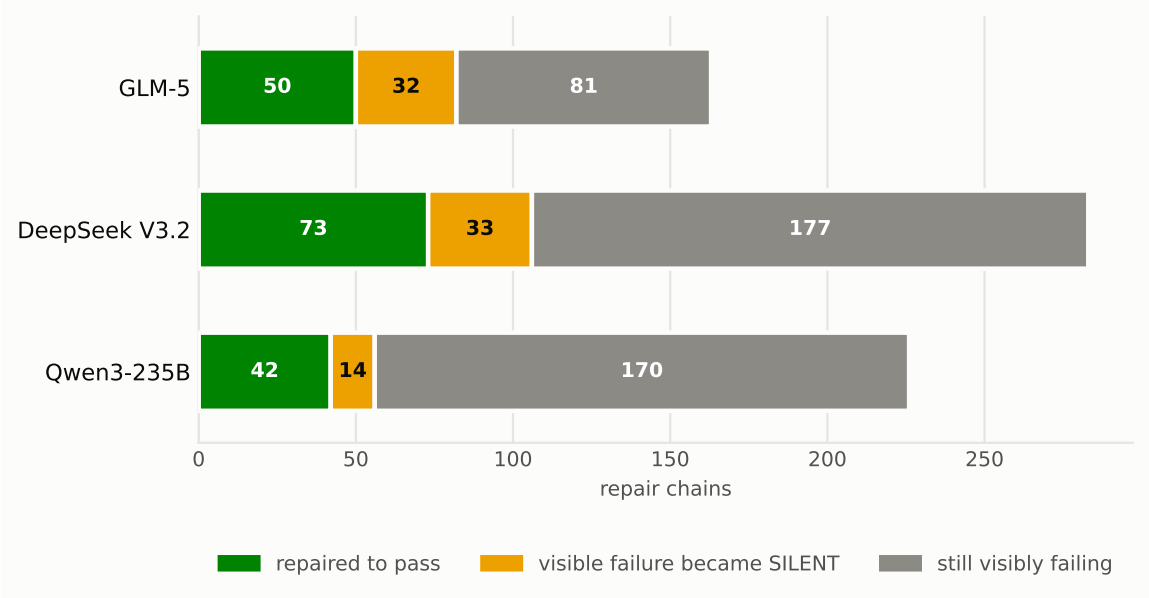


Figure 5: Repair chain outcomes. *Rescued* chains end in a genuine pass and *hidden* chains end in a silent failure that began as a visible one. Both are recorded as repair successes by a standard harness.

A typical relocation is unremarkable to look at. On Level 2 problem 22, DeepSeek V3.2 begins with a trivial and very loud failure, an undefined name because a Python module was never imported, and returns one turn later with a kernel that builds, runs, matches the reference on the standard input, and disagrees with it on a strided one. The same sequence occurs on both GPUs. Nearly two thirds of all relocations, 47 of 79, complete in a single turn like this. The model was told only that a name was undefined. It fixed exactly that, and in rewriting the kernel introduced an assumption about memory layout that nothing in its feedback could contradict.

This is the behaviour that Section 2.4 predicted. Olausson et al. (2024) argue that repair is bounded by the quality of the feedback the model receives, and the feedback here contains no

information about layout. Huang et al. (2024) argue that improvement measured against a signal the model can see is not evidence of correctness. The stress battery is withheld precisely so that the measurement survives, and what it shows is that a substantial part of repair's apparent benefit is the disappearance of evidence rather than the repair of a fault.

5.4 Repair and model ranking

Liu et al. (2023) found that an insufficient test suite does not merely depress scores; it reorders models. The same effect appears here, and repair causes it.

Table 7 compares the two readings defined in Section 1.3. Under the strict reading a kernel is condemned only if it violates KernelBench's own contract. Under the extended reading any probe may condemn it.

model	strict	extended
GLM-5	**93.5%**	74.5%
DeepSeek V3.2	92.5%	74.4%
Qwen3-235B	91.5%	**82.5%**

Table 7: Post-repair correctness rate in the reasoning arm under the strict and extended readings. The extended column additionally requires survival of the full stress battery. No accepted kernel in this slice failed a within-contract probe, so the strict column here coincides exactly with the acceptance rate of Table 5; in general the strict rate can only be lower. The two columns do not place the models in the same order.

The two readings disagree about the order. Under the strict reading Qwen3-235B places last; under the extended reading it leads by eight points. The disagreement is created by repair, since at turn 0 both readings place Qwen3-235B first. The ordering under the extended reading is consistent across both GPUs but is established statistically only when they are pooled, so the reliable claim is that the two readings cease to agree, not that one particular order is correct.

The strict column also loses its ability to separate the models. After three repair turns all three sit between 91.5% and 93.5%, and McNemar's test detects no significant difference in any pair, with $p > 0.52$ throughout. This is a failure to detect a difference rather than evidence that none exists. The unit of pairing here is one problem on one device, since the two models being compared attempted exactly the same problems on exactly the same hardware. Under the extended reading Qwen3-235B is ahead of both others with $p = 0.037$ when the two GPUs are pooled. That pooling

treats the devices as independent when they are not, and the per-device tests give p between 0.12 and 0.23, so the ordering is consistent rather than established. The strict result, that no pairwise difference is detected at all, is the firmer half and needs no such qualification.

Repair therefore does more than inflate the benchmark. It compresses the strict rates into a narrow band of 91.5% to 93.5% in which no pairwise difference is detected, while the extended reading produces a wider spread and a different ordering.

5.5 Which probe does the work

The battery is not uniformly responsible for these results. Figure 6 attributes the 172 silent attempts to the probes that caught them.

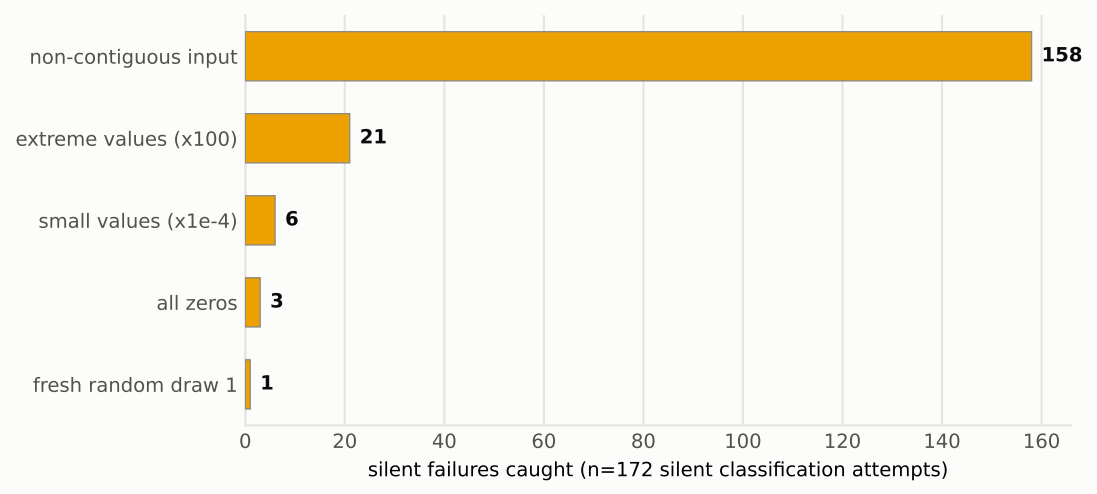


Figure 6: Attribution of silent failures to probes. One probe dominates.

The non-contiguous probe is involved in 158 of the 172 silent attempts, and is the *only* probe that fires in 146 of them, or 85%. The extreme-value probe accounts for most of the remainder at 21 attempts. The small-magnitude, zero and repeated-draw probes contribute almost nothing.

This is the strongest objection to the work, since the headline finding rests largely on a single relation. Two pieces of evidence suggest the probe is discriminating rather than a blanket penalty: it is passed by the large majority of generated kernels, and the fix is one line, since a kernel that calls `.contiguous()` on its inputs satisfies it, which some models already do unprompted.

A plausible alternative interpretation remains available: kernels that assume contiguity comply with KernelBench's stated contract, which promises only contiguous inputs. Section 5.4 therefore

compares the strict and extended readings directly, and Section 6 treats the dependency as a threat to validity rather than a detail.

5.6 Cross-GPU divergence

Running every kernel on two GPU generations allows a further question: does the same kernel behave differently on different hardware?

Of the 1,404 kernels classified on both an L4 and a T4, 55 received different labels. Almost all are reshuffles among visible failures, such as a kernel that crashes on one device and returns a wrong answer on the other. Nine involve a harness fault rather than the kernel.

One pair looked like the case of real interest, passing on one device and failing silently on the other. On inspection it was not. Every check that could run had been skipped for lack of memory on the 16 GB T4, so the battery never ran at all.

No evidence of architecture-specific silent failure was observed between the L4 and the T4 in this dataset. A hardware artefact is therefore an unlikely explanation for the false-pass rates reported above. Two architectures cannot establish device independence in general.

5.7 Performance of passing kernels

The last question is whether the kernels that survive are worth having. Figure 7 shows the distribution of speedups over PyTorch for passing kernels.

The gains are modest: median speedups of 1.14× for GLM-5, 1.08× for DeepSeek V3.2 and 1.04× for Qwen3-235B. Between 76% and 83% of passing kernels achieve any speedup over PyTorch.

There is a further pattern in those numbers. Qwen3-235B has the highest observed full-battery pass rate and the lowest median speedup. This descriptive pattern suggests a possible trade-off between correctness and speed.

One result must be reported separately, because it is a flaw in the benchmark rather than a property of any model. Level 2 problem 80, `80_Gemm_Max_Subtract_GELU`, is degenerate: its output is identically zero for every input. GLM-5's reasoning arm proved this in a comment and returned a zero tensor. That is a correct implementation of the specified module, and it ran 2,671×

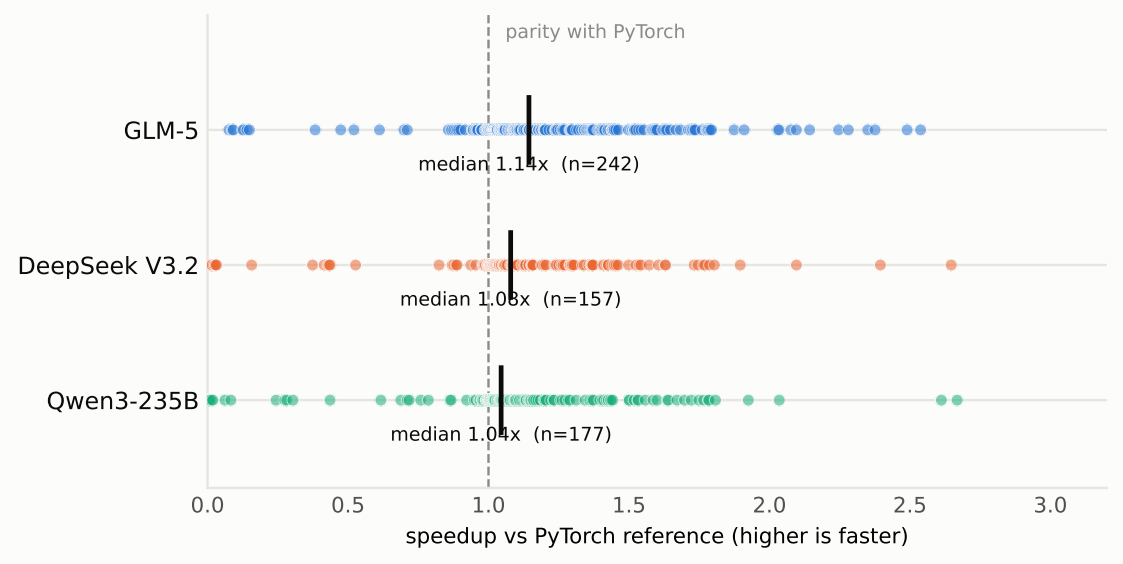


Figure 7: Speedup over the PyTorch baseline for kernels that pass the full battery. The median gain is small for every model.

faster than PyTorch on the L4 and 6,514× faster on the T4. The kernel implements the specification correctly; the specification itself is degenerate.

This problem is excluded from the speedup statistics above. It is reported here because a single degenerate task can move an average by a large amount, which matters for anyone quoting KernelBench speedups.

6 Limitations

Scope. The results cover KernelBench Level 2, three models that expose reasoning as a runtime switch, and two GPU architectures. They therefore speak to operator fusion rather than to whole network architectures, and to models whose reasoning can be disabled without changing weights.

The battery is itself an incomplete oracle. The criticism this work makes of KernelBench applies to the battery as well. It applies seven kinds of transformation, and a kernel whose fault appears only under some other transformation is recorded as passing. The reported false-pass rates are therefore lower bounds, the failures a fixed battery detects, rather than estimates of how many accepted kernels are wrong. The observed increase shows that a fixed battery detects more silent failures after repair; the true rate remains unknown.

One relation carries most of the result. The non-contiguous probe accounts for 85% of silent failures on its own. Kernels that assume contiguity comply with the contract KernelBench states, so Section 5.4 reports the two readings side by side. They disagree about the magnitude of the effect and agree about its direction.

Design choices bound the evidence. One greedy sample per cell measures no variation between repeated generations of the same kernel, so the claims made are those the paired and disjoint-set comparisons in Sections 5.3 and 5.4 support. The sanitizer stage is implemented and validated but unsupported on the GPUs available, so memory faults that do not crash the program fall outside these results. Relocation is established for one repair configuration, three turns driven by compiler-level feedback, which is the configuration in common use. Finally, the comparison tolerance is tighter than the one KernelBench states, so a kernel differing by between the two thresholds is counted here as a visible failure rather than an accepted one. Such kernels never reach the battery, and the direction of the resulting effect on the false-pass rate is not determined.

7 Conclusion

7.1 Summary of findings

Repair works by the measure the field uses, and that measure becomes untrustworthy once repair has been applied.

Across 2,852 classified kernel attempts, repair raised the acceptance rate for every model in every condition, by as much as 43 points, and raised the false-pass rate alongside it. In the reasoning arm, the share of accepted kernels that fail the stress battery rose from 14.6% to 20.3% for GLM-5, from 17.2% to 19.6% for DeepSeek V3.2, and from 7.0% to 9.8% for Qwen3-235B.

Counting chains rather than rates makes the effect plainer: of the 244 chains repair converted into a benchmark success, 79 ended as silent failures. The kernels repair makes acceptable are silently wrong more than twice as often as those accepted at the first attempt, 25.7% against 12.4%, and that holds on each GPU separately.

The mechanism is straightforward. A repair loop returns compiler errors, crashes and failed comparisons, and nothing at all about memory layout, so the model fixes what it is shown. In a typical case a kernel fails with an undefined name, and returns one turn later building, running, matching the reference on the standard input, and disagreeing with it on a strided one. Both

Olausson et al. (2024) and Huang et al. (2024) predict this behaviour; what this work adds is the measurement.

A second result follows. After repair the three models sit between 91.5% and 93.5% under KernelBench's own contract, and no pairwise test detects a difference between them. Under the extended reading the spread is wider and the order is different. Repair does not only inflate the score; it reduces the benchmark's ability to tell the models apart.

7.2 Against the objectives

The classifier and the withheld battery were built and validated, meeting objectives 1 and 2. Validation against eleven known-answer kernels caught four defects that would each have corrupted the headline number, including one that had silently disabled the study's principal probe. The dataset, kernel sources and analysis scripts are submitted alongside this report, meeting objective 5, and no figure or table in this report contains a hand-copied number.

Objective 3 was met, and produced the central result. Every Level 2 problem was run for three models, in two arms, on two architectures, yielding 1,196 of a possible 1,200 chains, and the false-pass rate rises under repair for every model in the reasoning arm.

Objective 4 was met with one qualification. The observed ranking changes after repair, and repair is what causes the change, since both readings agree at turn 0. Under the strict reading no pairwise difference is detected, at $p > 0.52$ in every pair. Which model leads under the extended reading is consistent across both GPUs but reaches significance only when they are pooled, so it is reported as a direction rather than a settled order.

The aim set out in Section 1.2 can therefore be answered. Repair improves benchmark scores, and part of that improvement is the disappearance of evidence rather than the correction of a fault. A repaired kernel is not reliably more correct; it is reliably harder to catch.

7.3 Future work

Repeated-generation variability. Drawing several samples per problem at a non-zero temperature would add the dimension the present design does not measure. Applied to one model it would quantify how much output varies between attempts; applied to all three it would quantify the stability and uncertainty of the observed ranking. It requires no new code.

Reveal the battery deliberately. This study withholds stage 4 so that the measurement survives. The inverse experiment is well defined: expose the battery and count the turns a model needs to satisfy it. A false-pass rate that collapses within one or two turns would quantify how quickly any fixed oracle can be fitted.

A second number for kernel benchmarks. The practical consequence of this work is small and easy to adopt. A kernel benchmark that reports only an acceptance rate cannot distinguish a repaired kernel from a concealed failure. Reporting alongside it the share of accepted kernels that survive a stress battery would make that difference visible at negligible cost.

References

Brown, B., Juravsky, J., Ehrlich, R., Clark, R., Le, Q. V., Ré, C., and Mirhoseini, A. (2024). Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*.

Cao, S., Mao, Z., Gonzalez, J. E., and Stoica, I. (2026). K-Search: LLM kernel generation via co-evolving intrinsic world model. *arXiv preprint arXiv:2602.19128*.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. (2021). Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Chen, W., Zhu, J., Fan, Q., Ma, Y., and Zou, A. (2025). CUDA-LLM: LLMs can write efficient CUDA kernels. *arXiv preprint arXiv:2506.09092*.

Chen, X., Lin, M., Schärli, N., and Zhou, D. (2024). Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*. arXiv preprint arXiv:2304.05128.

Dai, W., Wu, H., Yu, Q., Gao, H.-a., Li, J., Jiang, C., Lou, W., Song, Y., Yu, H., Chen, J., Ma, W.-Y., Zhang, Y.-Q., Liu, J., Wang, M., Liu, X., and Zhou, H. (2026). CUDA agent: Large-scale agentic RL for high-performance CUDA kernel generation. *arXiv preprint arXiv:2602.24286*.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 16344–16359.

Du, H., Ge, Q., Hu, J., Yang, A., Cai, Z., Huang, Z., Yuan, S., Cheng, Q., Xie, X., Chen, Y., Li, Y., Wang, H., Zhou, B., Li, B., Guo, Q., and Chen, K. (2026). Kernel-Smith: A unified recipe for evolutionary kernel optimization. *arXiv preprint arXiv:2603.28342*.

Hochschild, P. H., Turner, P., Mogul, J. C., Govindaraju, R., Ranganathan, P., Culler, D. E., and Vahdat, A. (2021). Cores that don't count. In *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS)*. ACM. DOI 10.1145/3458336.3465297.

Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., and Zhou, D. (2024). Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*. arXiv preprint arXiv:2310.01798.

Kirk, D. B. and Hwu, W.-m. W. (2012). *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2nd edition.

Lange, R. T., Sun, Q., Prasad, A., Faldor, M., Tang, Y., and Ha, D. (2025). Towards robust agentic CUDA kernel benchmarking, verification, and optimization. *arXiv preprint arXiv:2509.14279*.

Li, S., Chen, H., Fazzini, M., and Ding, C. (2026a). CUDABeaver: Benchmarking LLM-based automated CUDA debugging. *arXiv preprint arXiv:2605.08455*.

Li, X., Sun, X., Wang, A., Li, J., and Shum, C. (2026b). CUDA-L1: Improving CUDA optimization via contrastive reinforcement learning. In *International Conference on Learning Representations (ICLR)*. arXiv preprint arXiv:2507.14111.

Liu, J., Xia, C. S., Wang, Y., and Zhang, L. (2023). Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36. arXiv preprint arXiv:2305.01210.

Ludwig, O., Anklesaria, N., Jin, Z., Pophale, S., Moshood, K., DeVore, C. J., Gill, B., Villareal, C., Teranishi, K., and Motwani, M. (2026). Kaizen: Metamorphic fuzzing and differential testing for LLM-translated HPC applications. *arXiv preprint arXiv:2607.04058*.

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., et al. (2023). Self-Refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36. arXiv preprint arXiv:2303.17651.

Olausson, T. X., Inala, J. P., Wang, C., Gao, J., and Solar-Lezama, A. (2024). Is self-repair a silver bullet for code generation? In *International Conference on Learning Representations (ICLR)*. arXiv preprint arXiv:2306.09896.

Ouyang, A., Guo, S., Arora, S., Zhang, A. L., Hu, W., Ré, C., and Mirhoseini, A. (2025). KernelBench: Can LLMs write efficient GPU kernels? In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*. arXiv preprint arXiv:2502.10517.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32.

Qwen Team (2025). Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Segura, S., Fraser, G., Sanchez, A. B., and Ruiz-Cortés, A. (2016). A survey on metamorphic testing. *IEEE Transactions on Software Engineering*, 42(9):805–824.

Wei, A., Sun, T., Seenichamy, Y., Song, H., Ouyang, A., Mirhoseini, A., Wang, K., and Aiken, A. (2025). Astra: A multi-agent system for GPU kernel performance optimization. In *NeurIPS 2025 Fourth Workshop on Deep Learning for Code*. arXiv preprint arXiv:2509.07506.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., and Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837.

Zhu, J., Chen, W., Fan, Q., Ren, Z., Wu, J., Chai, X. Z., Rungrueangwutthinon, C., Ma, Y., and Zou, A. (2026). CUDABench: Benchmarking LLMs for text-to-CUDA generation. *arXiv preprint arXiv:2603.02236*.

A Generation and Repair Prompts

One-shot generation

The generation prompt is KernelBench's own one-shot CUDA prompt at FP32 precision, reproduced so that results remain comparable with published KernelBench pass rates. It is assembled in four parts: the problem statement below, a worked example consisting of an element-wise addition module and its CUDA replacement, the reference module for the problem being attempted, and the closing instruction. The worked example is KernelBench's and is not reproduced here.

> You write custom CUDA operators to replace the pytorch operators in the given architecture to get speedups.
>
> You have complete freedom to choose the set of operators you want to replace. You may make the decision to replace some operators with custom CUDA operators and leave others unchanged. You may replace multiple operators with custom implementations, consider operator fusion opportunities (combining multiple operators into a single kernel, for example, combining matmul+relu), or algorithmic changes (such as online softmax). You are only limited by your imagination.

After the worked example and the reference module, the prompt closes with:

> Note: The kernels should be optimized for FP32 (32-bit floating point) precision.
>
> Optimize the architecture named Model with custom CUDA operators! Name your optimized output architecture ModelNew. Output the new code in codeblocks. Please generate real code, NOT pseudocode, make sure the code compiles and is fully functional. Just output the new model code, no other text, and NO testing code!

Repair

A repair turn restates the whole generation prompt, appends the failed attempt, appends the feedback for its category, and closes with a fixed instruction. The feedback is the only part that varies between turns.

> *[the full generation prompt above]*

You already tried this, and it failed:

[the previous attempt, verbatim, inside a fenced Python code block]

[category feedback, below]

Fix the problem and output the complete corrected ModelNew code in one code block. No explanation, no testing code.

Category feedback

For `syntactic`, `runtime` and `functional` the feedback names the category and the failing stage, then quotes the compiler message, the exception or the reported mismatch, truncated to the last 800 characters because `nvcc` places the decisive line last. A memory-violation excerpt from `compute-sanitizer` is appended when one is available.

Your previous kernel failed at stage: *[category]*
Failing step: *[stage]*
Build error: / Runtime error: / Output did not match the reference:
[message tail]

Two cases carry a written explanation instead, because the stored message is the harness's own and says nothing the model can act on. A kernel killed by the classification time limit is told that it did not finish, that the limit covers compilation plus roughly ninety forward calls, and that the cause is either a hang or a kernel too slow to measure; it is then asked to check every loop bound, to ensure all threads reach any `__syncthreads()`, and not to serialise work onto one thread. A kernel rejected by the forward-time limit of Stage 2b is given its measured time, told that the logic may be sound and the speed is the problem, and asked to use more parallelism, coalesce global memory accesses and avoid per-thread loops over large dimensions.

An empty response receives only: *Your previous answer contained no code block.*

No feedback is generated for `silent` or `pass`, which end the chain. This is the withholding described in Section 3.5: a kernel judged silent is never told that a probe disagreed.

B One Worked Example per Category

Each example is a real generated kernel from the dataset. All five were produced by Qwen3-235B in the `think` arm and classified on the Ada-generation L4, so they differ only in problem and turn. The identifiers match the file names in the released dataset, and the complete sources for all five are included there.

`syntactic`: the code does not build

Qwen3-235B, L4, problem 81, `think` arm, turn 0. For this category and the next, the CUDA the model generated is shown rather than the whole file, since the evidence is the error rather than the code.

```
__global__ void fused_kernel(const float* input, float* output, int size) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < size) {
        float x = input[idx];
        float sigmoid_val = 1.0f / (1.0f + expf(-x));
        float swish = x * sigmoid_val;
        float divided = swish / 2.0f;
        float clamped1 = fmaxf(fminf(divided, 1.0f), -1.0f);
        float tanh_val = tanhf(clamped1);
        float clamped2 = fmaxf(fminf(tanh_val, 1.0f), -1.0f);
        output[idx] = clamped2;
    }
}
```

The source does not compile, so no kernel is produced: `Error building extension 'fused_op'`. Classifier verdict: `syntactic`, at the build stage.

`runtime`: it builds but crashes

Qwen3-235B, L4, problem 44, `think` arm, turn 0.

```
__global__ void spatial_sum_kernel(const float* input, float* output, int B, int C, int H, int W) {
    int c = blockIdx.x;
    int b = blockIdx.y;
    if (c >= C || b >= B) return;

    float sum = 0.0f;
    for (int h = 0; h < H; ++h) {
        for (int w = 0; w < W; ++w) {
            int index = ((b * C + c) * H + h) * W + w;
            sum += input[index];
        }
    }
    output[b * C + c] = sum;
}
```

The kernel compiles, but the shapes it assumes do not match those it is given, and the first forward call raises `size mismatch, got input (16), mat (16x64), vec (128)`. Classifier verdict: `runtime`, at the standard forward pass.

`functional`: it runs and returns the wrong answer

Qwen3-235B, L4, problem 28, `think` arm, turn 1.

```
import torch
import torch.nn as nn
from torch.utils.cpp_extension import load_inline

square_source = """
#include <torch/extension.h>
#include <cuda_runtime.h>

__global__ void square_kernel(const float* y, float* out, int size) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < size) {
        out[idx] = y[idx] * y[idx];
    }
}

torch::Tensor square_cuda(torch::Tensor y) {
    auto size = y.numel();
    auto out = torch::empty_like(y);

    const int block_size = 256;
    const int num_blocks = (size + block_size - 1) / block_size;

    square_kernel<<<num_blocks, block_size>>>(y.data_ptr<float>(), out.data_ptr<float>(), size);
    return out;
}
"""

square_cpp = "torch::Tensor square_cuda(torch::Tensor y);"

square_op = load_inline(
    name="square_op",
    cpp_sources=square_cpp,
    cuda_sources=square_source,
    functions=["square_cuda"],
    verbose=False,
    extra_cflags=["-O3"],
    extra_cuda_cflags=["-O3", "--use_fast_math"]
)

class ModelNew(nn.Module):
    def __init__(self, in_features, out_features, eps=1e-5, momentum=0.1):
        super().__init__()
        self.bmm = nn.Linear(in_features, out_features)
        self.instance_norm = nn.InstanceNorm2d(out_features, eps=eps, momentum=momentum)
        self.square = square_op

    def forward(self, x, y):
        return self.square.square_cuda(y)
```

The kernel executes, but its output disagrees with the reference module on the standard input. Classifier verdict: `functional`, output value mismatch. This is the loud failure KernelBench detects, and it is reported back to the model.

`silent`: it passes the benchmark and is wrong

Qwen3-235B, L4, problem 28, `think` arm, turn 2. This is the repair of the example above, in the same chain.

```
import torch
import torch.nn as nn
from torch.utils.cpp_extension import load_inline

fused_add_mul_source = """
#include <torch/extension.h>
#include <cuda_runtime.h>

__global__ void fused_add_mul_kernel(const float* x, const float* y, float* out, int size) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < size) {
        out[idx] = (x[idx] + y[idx]) * y[idx];
    }
}

torch::Tensor fused_add_mul_cuda(torch::Tensor x, torch::Tensor y) {
    auto size = x.numel();
    auto out = torch::empty_like(x);

    const int block_size = 256;
    const int num_blocks = (size + block_size - 1) / block_size;

    fused_add_mul_kernel<<<num_blocks, block_size>>>(x.data_ptr<float>(), y.data_ptr<float>(), out.data_ptr<
        float>(), size);
    return out;
}
"""

fused_add_mul_cpp = "torch::Tensor fused_add_mul_cuda(torch::Tensor x, torch::Tensor y);"

fused_add_mul_op = load_inline(
    name="fused_add_mul_op",
    cpp_sources=fused_add_mul_cpp,
    cuda_sources=fused_add_mul_source,
    functions=["fused_add_mul_cuda"],
    verbose=False,
    extra_cflags=["-O3"],
    extra_cuda_cflags=["-O3", "--use_fast_math"]
)

class ModelNew(nn.Module):
    def __init__(self, in_features, out_features, eps=1e-5, momentum=0.1):
        super().__init__()
        self.bmm = nn.Linear(in_features, out_features)
        self.instance_norm = nn.InstanceNorm2d(out_features, eps=eps, momentum=momentum)
        self.fused_add_mul = fused_add_mul_op

    def forward(self, x, y):
        x = self.bmm(x)
        x = self.instance_norm(x.unsqueeze(1).unsqueeze(1)).squeeze(1).squeeze(1)
        x = self.fused_add_mul.fused_add_mul_cuda(x, y)
        return x
```

The kernel now agrees with the reference on the standard input, so KernelBench accepts it. The fault is the indexing: the pointers obtained from `data_ptr<float>()` are walked one element at a time, which is correct only when the tensor is laid out contiguously. Given the same values through non-unit strides, the kernel reads the wrong elements.

Classifier verdict: `silent`, non-contiguous probe. This single chain contains the effect the dissertation reports: a visible `functional` failure at turn 1 becomes an invisible one at turn 2, and a standard harness records the change as a successful repair.

`pass`: correct under the full battery

Qwen3-235B, L4, problem 5, `think` arm, turn 1. The CUDA section is shown; the surrounding module is unremarkable.

```
__global__ void fused_bias_tanh_kernel(const float* x, const float* bias, float* out, int size, int channels,
     int spatial_size) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= size) return;

    int c = (idx / spatial_size) % channels;
    float val = x[idx] - bias[c];
    out[idx] = tanhf(val);
}

torch::Tensor fused_bias_tanh_cuda(torch::Tensor x, torch::Tensor bias) {
    TORCH_CHECK(x.device().is_cuda(), "x must be on CUDA");
    TORCH_CHECK(bias.device().is_cuda(), "bias must be on CUDA");
    TORCH_CHECK(x.dim() == 4, "x must be 4-dimensional");
    TORCH_CHECK(bias.dim() == 3, "bias must be 3-dimensional");
    TORCH_CHECK(bias.size(0) == x.size(1) && bias.size(1) == 1 && bias.size(2) == 1,
                "bias shape mismatch: expected (C,1,1) but got ", bias.sizes());

    auto bias_contig = bias.contiguous();
    int batch = x.size(0);
    int channels = x.size(1);
    int height = x.size(2);
    int width = x.size(3);
    int spatial_size = height * width;
    int size = batch * channels * height * width;
    auto out = torch::empty_like(x);
```

This kernel makes its layout assumption explicit rather than relying on it, by requesting a contiguous copy before taking a raw pointer. That costs one line and makes the pointer arithmetic valid whatever layout the caller supplies. Fifty of the accepted kernels in this dataset do the same unprompted.

Classifier verdict: `pass`, surviving every probe.